CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 13: Graph Coloring

Harvard SEAS - Spring 2026 *2026-03-23*

1 Announcements

- Pset 5 due Thursday
- Pset 6 release on Tuesday
- SRE 4 on Tuesday

Recommended Reading:

- Hesterberg–Vadhan Ch. 13.3, Ch 12.
- Roughgarden III, 13.1
- Lewis & Zax 18

2 Greedy algorithm for Interval Scheduling

In the last lecture, we gave a greedy algorithm for finding large independent sets in a graph and found that the independent sets may not be the largest. However, we can do much better for Interval Scheduling–Optimization, and find the true largest independent set. The point is that the conflict graphs that arise from collections of intervals are much more structured than arbitrary graphs.

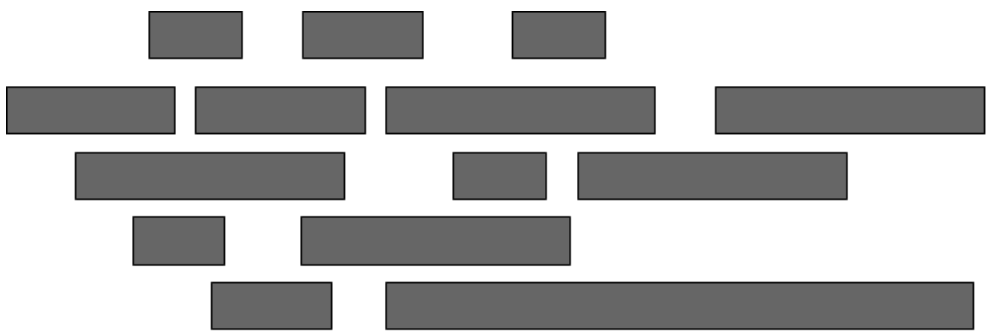
Specifically, a nearly linear-time greedy algorithm for Interval Scheduling–Optimization is given in Algorithm 2.1.

```
GreedyIntervalScheduling(x):
Input  : A list x of n intervals [a, b], with a, b ∈ ℚ
Output : A "large" subset of the input intervals that are disjoint
         from each other
Sort the intervals with                    ;
S = ∅;
foreach i = 0 to n − 1 do
    |                                      ;
    return S
```

Algorithm 2.1: `GreedyIntervalScheduling()`

Theorem 2.1. *For every input list x of intervals,* `GreedyIntervalScheduling(x)` *finds an optimal solution to* Interval Scheduling–Optimization *in time $O(n \log n)$.*

Proof. For correctness, we will show that for any Interval Scheduling–Optimization instance (such as the one shown in black in below) and any solution $S^* = \{i_0^* \leq i_1^* \leq \ldots \leq i_{k^*-1}^*\}$ (where $i_j^* \in S^*$ means that we include the interval $[a'_{i_j^*}, b'_{i_j^*}]$), we can modify S^* into the greedy solution $S = \{i_0, i_1, \ldots, i_{k-1}\}$ by "smushing it left."



Induction Hypothesis. Formally, we can prove the following by induction on $j = 0, 1, \ldots, k^* - 1$:

1. The greedy solution contains at least $j + 1$ intervals.
2. $b'_{i_j} \leq b'_{i_j^*}$, i.e. the jth interval selected by the greedy algorithm ends no later than the jth interval in the optimal solution ("greedy stays ahead").

Setting $j = k^* - 1$ in Item 1, we see that `GreedyIntervalScheduling` finds a solution containing at least k^* intervals. That is, for every independent set S^*,

`GreedyIntervalScheduling` finds an independent set with at least as many intervals as S^*. Thus, `GreedyIntervalScheduling` finds an optimal solution.

Runtime.

□

3 Motivating graph coloring: register allocation

When covering RAM simulations, we saw how to simulate programs that many use many variables with ones that use a fixed number c of registers, by swapping variables in and out of memory.

New Approach. Avoid swapping in and out of memory, by reusing the same registers for different variables. (Most variables generated by compilers are temporary.)

This approach proceeds as follows: at each line of code, every 'live' temporary variable is assigned to one of the c registers. We need to ensure that no register is assigned to more than one live variable at a time. For each temporary variable `var`, we define a *live region* R, which are the lines of code in which the value of `var` needs to be maintained.

Example 3.1. Consider the (Word-)RAM program in Algorithm 3.1.

```
   SquaredDistanceToNegativeOne(x):
   Input        : An array x = (x[0], x[1], ..., x[n-1])
   Output       : (x[0]+1)^2 + (x[1]+1)^2 + ··· + (x[n-1]+1)^2
   Variables    : input_len, output_len, output_ptr, temp_0, temp_1, temp_2, temp_3
 0 output_ptr = input_len;
 1 output_len = 1;
 2 temp_3 = 0;
 3     IF input_len == 0 GOTO 15;
 4     temp_0 = 1;
 5     temp_0 = temp_0 + temp_3;
 6     input_len = input_len − temp_0;
 7     temp_1 = M[input_len];
 8     temp_1 = temp_1 + temp_0;
 9     temp_1 = temp_1 × temp_1;
10     temp_2 = M[output_ptr];
11     temp_2 = temp_2 + temp_1;
12     temp_3 = 0;
13     M[output_ptr] = temp_2;
14     IF temp_3 == 0 GOTO 3;
15 HALT ;                                  /* not an actual command */
```

Algorithm 3.1: Toy (Word-)RAM program, which interprets the input as the coordinates of a point in n dimensions and calculates its squared distance to the point $(-1, -1, \ldots, -1)$.

In this program, the live regions for the temporary variables $\texttt{temp}_0, \texttt{temp}_1, \texttt{temp}_2, \texttt{temp}_3$ are:

$$\begin{aligned} R_0 &= \\ R_1 &= \\ R_2 &= \\ R_3 &= \end{aligned}$$

We can model this problem graph-theoretically by defining a *conflict* graph (aka the "register interference graph"):

Graph coloring is the way to formulate the problem of finding a valid assignment of live regions to registers.

Definition 3.2. For an undirected graph $G = (V, E)$, a (proper[1]) *k-coloring* of G is

The computational problem of finding a coloring is called the Graph Coloring problem:

Input: A graph $G = (V, E)$ and a natural number k
Output: A k-coloring of G (if one exists)

Computational Problem Graph Coloring

The fastest known algorithms for Graph Coloring still run in exponential time. In SRE4, you will see an algorithm for $k = 3$ that runs in time

$$O\left((n+m)\binom{n}{n/3}\right) = O(1.89^n).$$

Returning to the Register Allocation problem, if we have a proper k-coloring f of the conflict graph, then we can safely replace each variable `var` with a new register (i.e. variable) $\texttt{reg}_{f(\texttt{var})}$, thereby using only the k variables $\texttt{reg}_0, \texttt{reg}_1, \ldots, \texttt{reg}_{k-1}$ in our new (but equivalent) program.

Using a 2-coloring of the conflict graph for our Toy (Word-)RAM program above, we obtain a new program after register allocation as follows:

Remark. A common variant of the Graph Coloring problem is to find a coloring using as *few* colors as possible, for a given a graph G. This problem is in some sense the opposite of another problem we recently looked at:

[1] An *improper* coloring allows us to assign the same color to vertices that share an edge, but in graph theory, 'coloring' means 'proper coloring' unless explicitly stated otherwise.

4 Greedy Coloring

A natural first attempt at graph coloring is to use a greedy strategy:

```
GreedyColoring(G):
Input  : A graph G = (V, E)
Output : A coloring f of G using "few" colors
0 Select an ordering v_0, v_1, v_2, ..., v_{n-1} of V;
1 foreach i = 0 to n - 1 do
2 |   f(v_i) = ____________________________________.
3 return f
```

Algorithm 4.1: A greedy coloring algorithm.

An example of this algorithm is depicted in Figure 1.

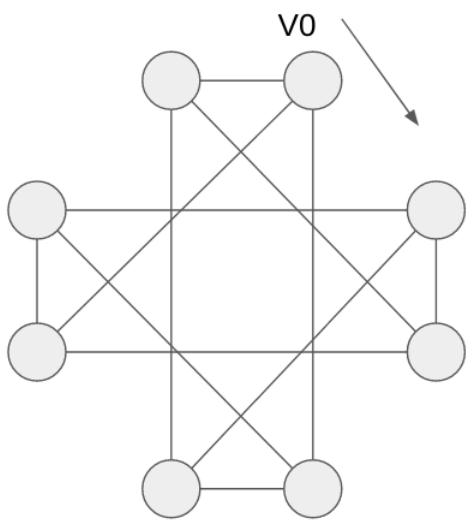


Figure 1: `GreedyColoring` algorithm with the ordering of vertices in clockwise direction starting from v_0

Assuming that we select the ordering (Line 0) in a straightforward manner (e.g. in the same order that the vertices are given in the input), `GreedyColoring`(G) can be implemented in time $O(n + m)$.

By inspection, `GreedyColoring`(G) always outputs a proper coloring of G. What can we prove about how many colors it uses?

Theorem 4.1. *When run on a graph $G = (V, E)$ with any ordering of vertices,* `GreedyColoring`(G) *will use at most* $\deg_{max}+1$ *colors, where* $\deg_{max} = \max\{\deg(v) : v \in V\}$.

Proof.

□

Remark. Note that this is an algorithmic proof of a pure graph theory fact: every graph is $(\deg_{max} + 1)$-colorable. However, this bound of $\deg_{max} + 1$ can be much larger than the number of colors actually needed to color G.

The performance of greedy algorithms can be very sensitive to the order in which decisions are made, and often we can achieve much better performance by picking a careful ordering. For example, we can process the vertices in *BFS order*, as described below.

```
BFSColoring(G):
Input  : A connected graph G = (V, E)
Output : A coloring f of G using "few" colors
Fix an arbitrary start vertex v_0 ∈ V;
Start breadth-first search from v_0 to obtain a vertex order v_1, v_2, ..., v_{n-1};
foreach i = 0 to n − 1 do
    f(v_i) = min {c ∈ ℕ : c ≠ f(v_j)  ∀j < i s.t. {v_i, v_j} ∈ E}.
return f
```

Algorithm 4.2: Greedy coloring in BFS order.

Running the `BFSColoring` algorithm on the same example as above, we obtain

Now, we show that, in general, for 2-colorable graphs, `BFSColoring` finds a 2-coloring efficiently.

Theorem 4.2. *If G is a connected 2-colorable graph, then* `BFSColoring`(G) *will color G using 2 colors.*

Proof sketch.

□

Corollary 4.3. Graph 2-Coloring *can be solved in time* $O(n + m)$.

Proof.

□